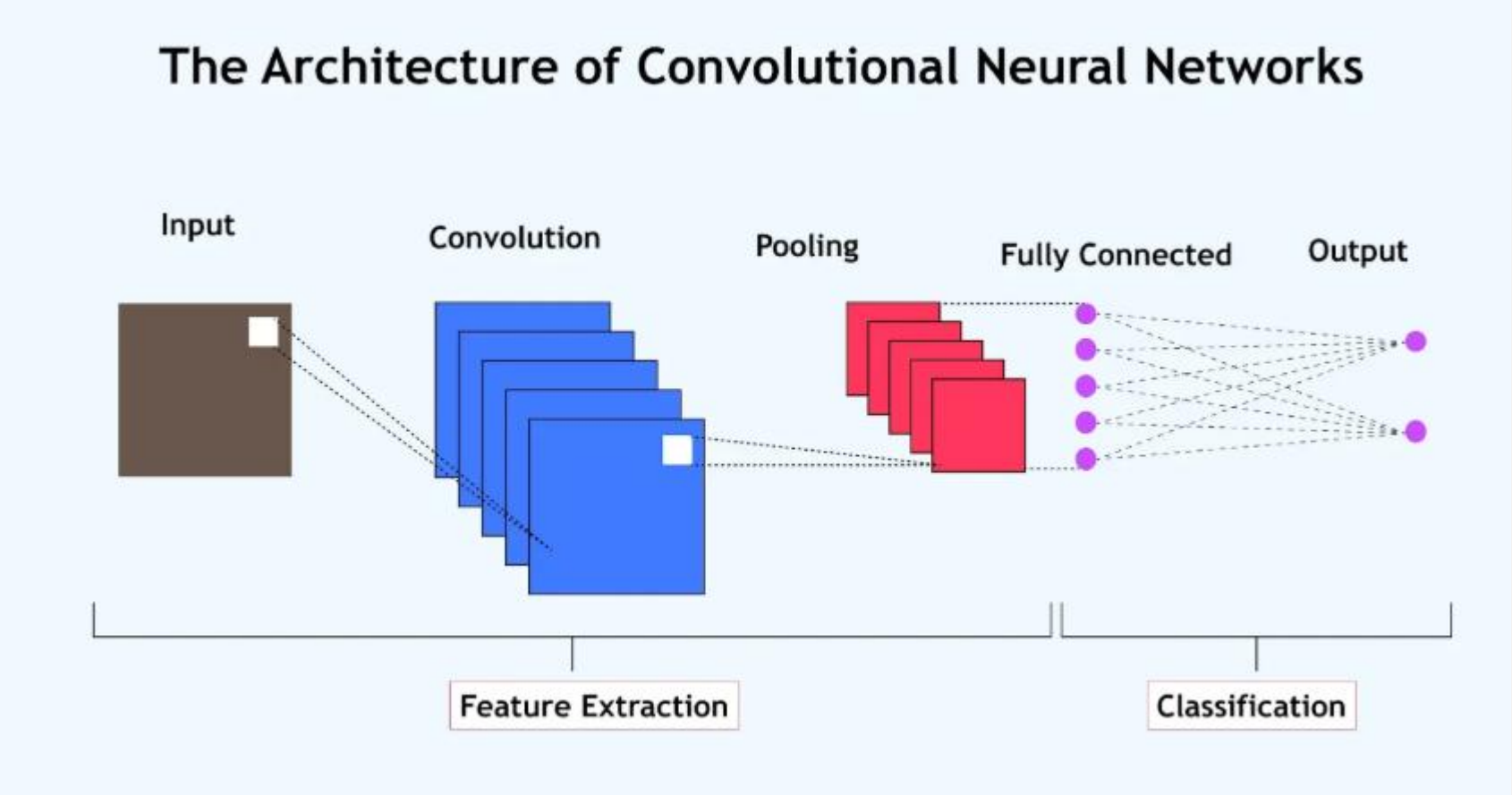
CNN ARCHITECTURE AND CNN FOR TEXT CLASSIFICATION

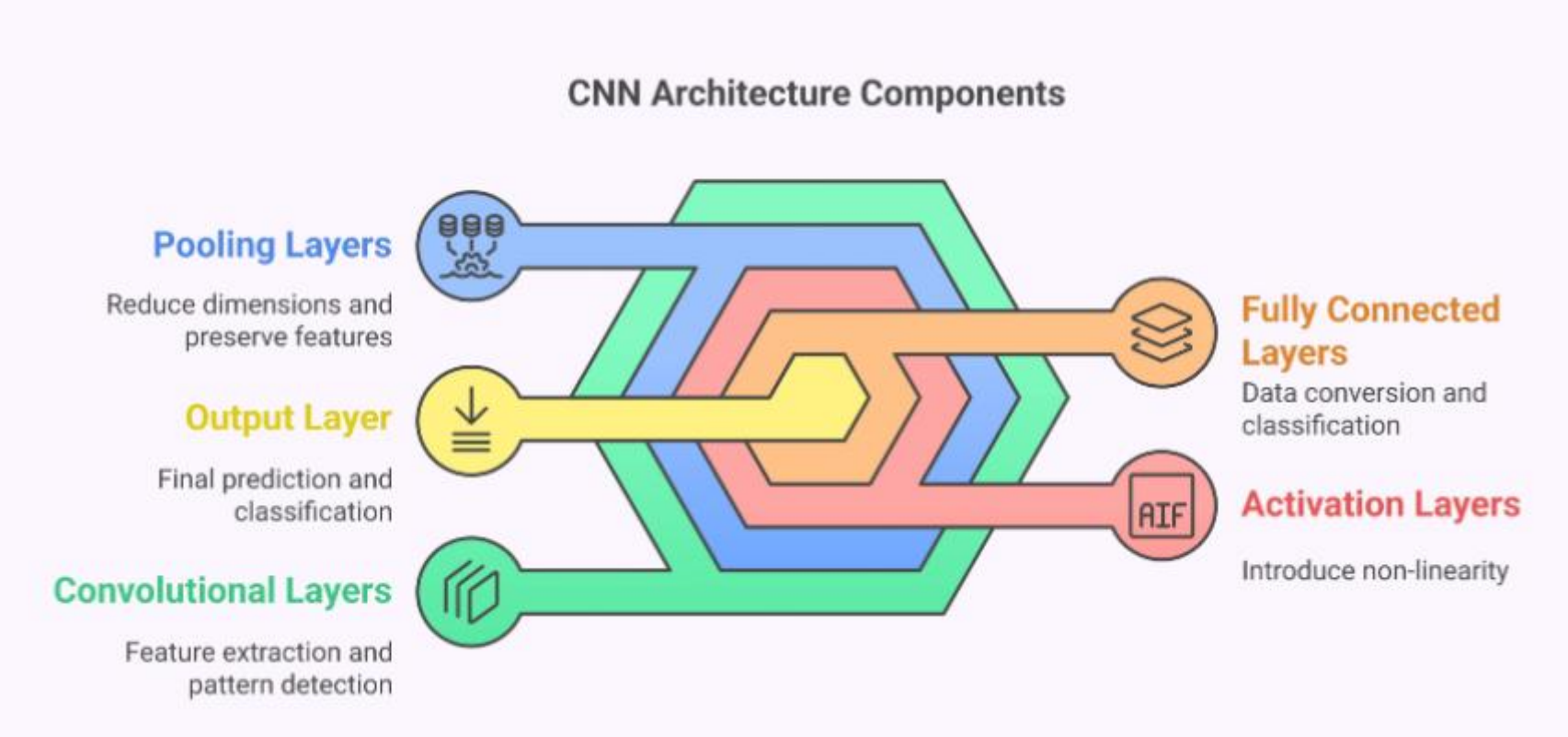
Presented By: Deena Fathimma

CNN Architecture

Convolutional Neural Networks (CNNs) are a type of deep learning model used for image recognition, processing, and classification. With a basic CNN architecture, it automatically and efficiently extracts features from input data.



The CNN architecture can be divided into five components.



Comprehensive Overview of the 5 Key Layers in CNN Architecture

1. Convolutional layer
2. Pooling layer
3. Fully connected layer,
4. Dropout layer,
5. Activation functions

work together in CNNs to extract features and classify data efficiently.

1. Convolutional Layer

This layer is the first layer that is used to extract the various features from the input images.

In this layer, the mathematical operation of convolution is performed between the input image and a filter of a particular size M x M.

By sliding the filter over the input image, the dot product is taken between the filter and the parts of the input image with respect to the size of the filter (M x M).

The output is termed as the Feature map which gives us information about the image such as the corners and edges.

Later, this feature map is fed to other layers to learn several other features of the input image.

The convolution layer in CNN passes the result to the next layer once applying the convolution operation in the input.

Convolutional layers in CNN are beneficial as they ensure the spatial relationship between the pixels is intact.

2. Pooling Layer

In most cases, a Convolutional Layer is followed by a Pooling Layer.

The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs.

This is performed by decreasing the connections between layers and independently operates on each feature map.

Depending upon method used, there are several types of Pooling operations. It basically summarises the features generated by a convolution layer.

In Max Pooling, the largest element is taken from feature map.

Average Pooling calculates the average of the elements in a predefined sized image section.

The total sum of the elements in the predefined section is computed in Sum Pooling.

The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer.

The primary aim of this layer is to decrease the size of the convolved feature map to reduce the computational costs.

This is performed by decreasing the connections between layers and independently operates on each feature map.

Depending upon method used, there are several types of Pooling operations. It basically summarises the features generated by a convolution layer.

In Max Pooling, the largest element is taken from feature map.

Average Pooling calculates the average of the elements in a predefined sized image section.

The total sum of the elements in the predefined section is computed in Sum Pooling.

The Pooling Layer usually serves as a bridge between the Convolutional Layer and the FC Layer.

3. Fully Connected Layer

The Fully Connected (FC) layer consists of weights and biases, along with the neurons, and is used to connect neurons across different layers.

These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

In this, the input image from the previous layers is flattened and fed to the FC layer.

The flattened vector then undergoes a few more FC layers, where the mathematical function operations usually take place.

In this stage, the classification process begins to take place.

The reason two layers are connected is that two fully connected layers will perform better than a single connected layer.

These layers in CNN reduce the human supervision.

4. Dropout

When all the features are connected to the FC layer, it can cause overfitting in the training dataset.

Overfitting occurs when a particular model works so well on the training data, causing a negative impact on the model's performance when used on the data.

To overcome this problem, a dropout layer is utilised wherein a few neurons are dropped from the neural network during the training process, resulting reduced size of the model.

On passing a dropout of 0.3, 30% of the nodes are dropped out randomly from the neural network.

Dropout results in improving the performance of a machine learning model as it prevents overfitting by making the network simpler.

It drops neurons from the neural networks during training.

5. Activation Functions

Finally, one of the most important parameters of the CNN model is the activation function.

They are used to learn and approximate any kind of continuous and complex relationship between the variables of the network.

In simple words, it decides which information of the model should fire in the forward direction and which ones should not at the end of the network.

It adds non-linearity to the network. There are several commonly used activation functions such as the ReLU, Softmax, tanH and the Sigmoid functions.

Each of these functions has a specific usage. For a binary classification CNN model, sigmoid and softmax functions are preferred for a multi-class classification; generally, softmax is used.

In simple terms, activation functions in a CNN model determine whether a neuron should be activated or not.

It decides whether the input to the work is important or not to predict using mathematical operations.

CNNs for Text Classification

While CNNs were initially developed for image processing, they have proven very effective for NLP tasks, especially text classification.

Text classification involves assigning predefined categories or labels to unstructured text documents.

This supervised learning task requires training models on labelled datasets where each document has a known category.

It transforms human-readable text into numerical representations that machine learning algorithms can process.

Why use CNN-based text classification?

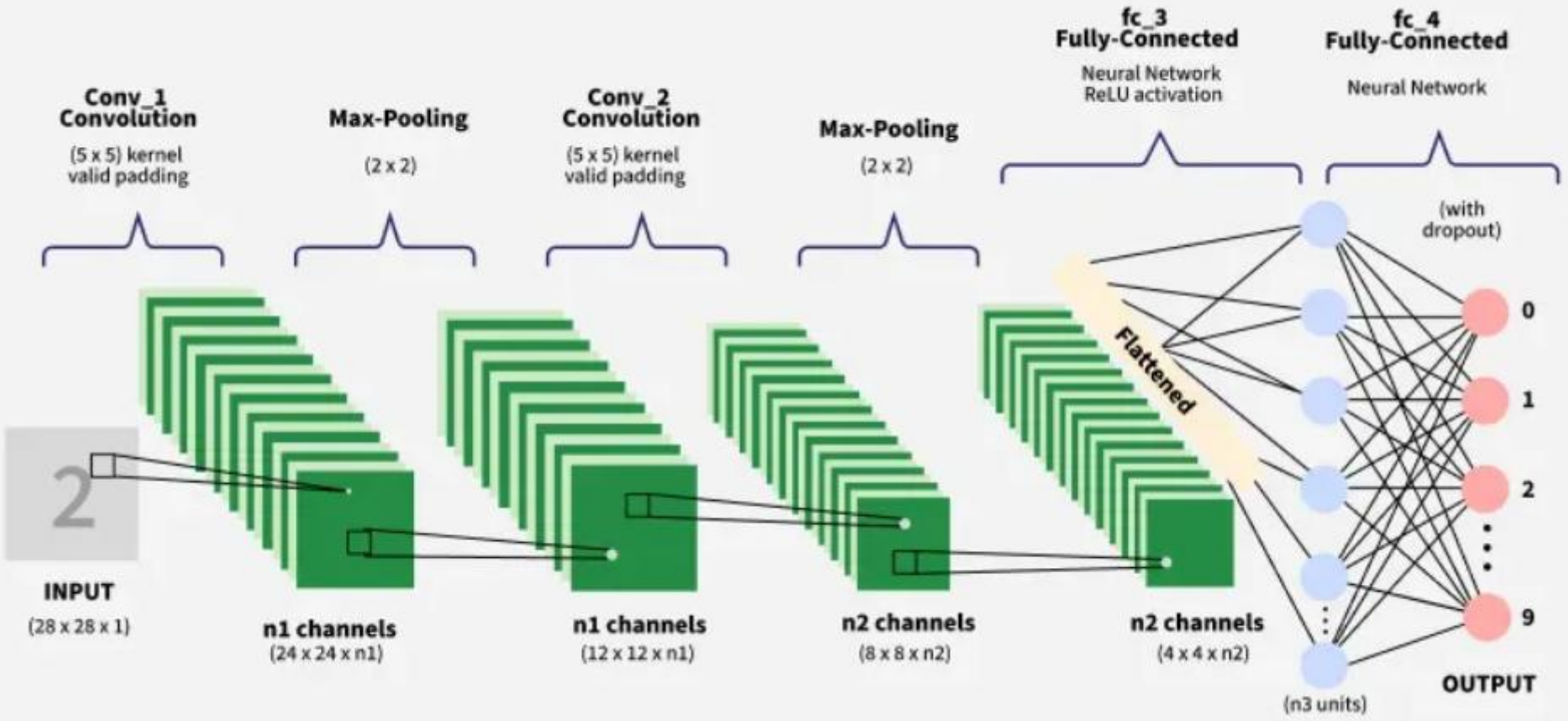
- Automatic feature extraction from raw text.
- Ability to capture local text patterns and n-gram features.
- Robust performance across various text classification tasks.
- Less preprocessing is required compared to traditional methods.

CNN Architecture for Text Processing

Convolutional Neural Networks adapt to text by treating documents as sequences of words rather than spatial images.

This adaptation requires modifications to traditional CNN architectures while preserving the core convolution and pooling operations.

- **Embedding Layer**: Converts words to dense vector representations
- **Convolutional Layers**: Apply filters to detect local text patterns
- **Pooling Layers**: Reduce dimensionality while preserving important features
- **Fully Connected Layers**: Combine features for final classification
- **Output Layer**: Produces probability distributions over target classes


Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with
dropout)
2
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
Flattened
n2 channels
(4 x 4 x n2)
(n3 units)
0
1
2
9
OUTPUT

CNNs For NLP In TensorFlow

Text Classification using CNN in TensorFlow

Step-by-Step Procedures

1. **Importing Libraries**

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropout
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
```

2. Loading Data

```
vocab_size = 10000
max_length = 500
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)
x_train = sequence.pad_sequences(x_train, maxlen=max_length)
x_test = sequence.pad_sequences(x_test, maxlen=max_length)
```

3. Building CNN model

```
model = Sequential([
    Embedding(vocab_size, 100, input_length=max_length),
    Conv1D(filters=128, kernel_size=5, activation='relu'),
    GlobalMaxPooling1D(),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
```

4. Compiling and Training the Model

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, batch_size=32, epochs=5, validation_split=0.2)
```

5. Evaluating the Model

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

FOR MORE REFERENCES, GO THROUGH THE LINK GIVEN BELOW

https://www.geeksforgeeks.org/nlp/text-classification-using-cnn/